Documento de arquitetura do sistema

UniMove | Grupo 05 | Universidade Católica de Brasília

Versão documental 2.0 • 13/11/2026

1. Objetivo

Este documento apresenta a arquitetura do UniMove, incluindo tecnologias, módulos, dados e integrações.

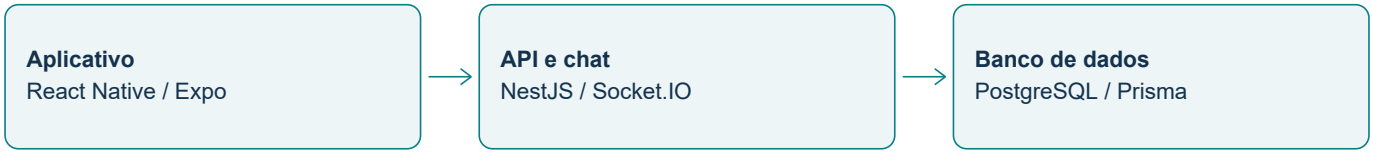
O sistema conecta estudantes motoristas e passageiros para compartilhar deslocamentos. O backend também oferece operações administrativas. A interface completa do painel administrativo não está neste checkout; existe apenas a tela de revisão acadêmica no aplicativo.

2. Organização e tecnologias

Camada	Implementação observada	Local
Aplicativo	React 19.3, React Native 0.88, Expo 58 e TypeScript	UniMove-front
API	NestJS 11.2.6, Node.js 24 e TypeScript	UniMove-back
Persistência	PostgreSQL, Prisma 7 e adapter pg	UniMove-back/prisma
Chat	Socket.IO 4.8, namespace /chat, transporte WebSocket	modules/chat
Mapas	MapLibre GL 5.6.1 com estilo OpenFreeMap	OpenFreeMap.tsx
Entregas externas	Expo Push, SMTP/Nodemailer e SMS/Twilio	modules/notifications
Publicação preparada	API no Render; PostgreSQL 17 nos workflows de teste	render.yaml e .github/workflows

As versões são as declaradas nos manifests, não uma garantia de versão instalada em produção. Não há Flutter/Dart, TypeORM, API biométrica ou infraestrutura AWS/Azure implementados neste checkout. Localizações possuem latitude e longitude; o schema atual não utiliza Geography/PostGIS como requisito de matching.

3. Estrutura lógica



HTTP /api/v1 e WebSocket /chat · Integrações externas condicionais: mapas, push, SMTP e SMS.

O backend é um monólito modular NestJS com apresentação, regras e persistência. Os módulos compartilham o PrismaService. Há services dedicados e também controllers que consultam Prisma diretamente; não existe uma camada uniforme de repositories para todos os domínios.

Fluxo principal: aplicativo → API HTTP /api/v1 → guards e validação → controller/service → Prisma/SQL → PostgreSQL. O chat usa um segundo canal: aplicativo → Socket.IO /chat → autorização da conversa → persistência → transmissão para a sala.

O aplicativo usa componentes funcionais, hooks e AuthContext. App.tsx seleciona as telas por estado e exige completar o perfil após autenticação. As integrações HTTP ficam em src/api e src/services. Componentes reutilizáveis e tema ficam em src/components e src/theme.ts. A implementação oferece caminhos Android, iOS e web; não houve certificação de execução em todos esses ambientes nesta revisão.

4. Módulos e responsabilidades

Domínio	Responsabilidade
auth	Cadastro, política de senha/aceite, login, refresh, logout e MFA
users	Perfil, foto, dados pessoais, preferências, sessões, exportação, exclusão, conquistas e indicadores
verification	Leitura do comprovante acadêmico, fila e decisão administrativa
vehicles	Veículos, foto da CNH e associação ao papel DRIVER
route-templates e matching	Rotas recorrentes e geração/consulta de caronas compatíveis
rides e ride-requests	Oferta avulsa, busca, detalhes, pedidos, aceitação, recusa e ciclo da viagem
chat	Conversas entre motorista e passageiro elegível, histórico e mensagens
notifications	Histórico no aplicativo, contagem, leitura, categorias e canais externos
ratings e operations	Avaliações, ocorrências e histórico com resumo financeiro estimado
admin	Sessão do painel, gestão de usuários, ocorrências, relatórios, auditoria e IAM
health	Liveness e consulta de prontidão do banco

5. Dados e transações

O schema modela usuários, papéis, sessões, veículos, CNH, fotos de perfil, verificação acadêmica, locais, rotas, viagens, pedidos, reservas, conversas, mensagens, cancelamentos, avaliações, notificações, preferências, bloqueios e auditoria. O IAM acrescenta AdminAccess, AdminInvite, AdminPasswordReset e AdminMfaRecovery.

Pedidos pendentes válidos retêm vagas no cálculo de disponibilidade. O pedido é individual, com seats igual a 1. Bloqueios de linhas e transações serializam operações de reserva. A aceitação cria a reserva e ajusta as vagas. O prazo padrão do pedido é 15 minutos, limitado pelo horário de partida.

O último administrador é protegido por invariante compartilhado em operações críticas. A exclusão da conta remove identificadores e documentos, invalida sessões e preserva relações operacionais com UUID técnico. Isso caracteriza pseudonimização dos registros retidos; não se promete apagamento físico de toda a história.

Migrações versionadas abrangem consentimento, perfil obrigatório, CNH, coordenadas, dados pessoais, verificação acadêmica, preferências, recibos de leitura, IAM e a central de notificações de 13/11/2026. A existência de uma migration no Git não comprova sua aplicação no banco publicado.

6. Busca e matching

O aplicativo consulta GET /rides. O servidor filtra por nomes de origem/destino sem diferenciar acentos e caixa, data DD/MM/AAAA, hora HH:mm exata e timezone IANA. A ordenação é por partida e ID; avaliações são exibidas como informação e não determinam o ranking. A resposta inclui items, total, page, limit e hasPendingRequest.

GET /matching/search é um fluxo distinto de rotas recorrentes. Usa IDs exatos de origem/destino, dia da semana, vagas e tolerância de horário configurável, padrão 30 minutos. Pode materializar viagens por upsert; portanto essa consulta tem efeito de escrita. A materialização utiliza offset -03:00, mesmo existindo timezone no modelo da rota. Não há cálculo de proximidade de 1 km, desvio viário ou ajuste por precisão GPS.

7. Identidade, autorização e privacidade

O cadastro valida dígitos verificadores do CPF, unicidade e domínio institucional configurado. Isso não consulta a Receita Federal nem comprova posse do e-mail. O vínculo acadêmico depende do PDF digital compatível e de revisão humana do GUID no portal oficial. O PDF é processado em memória e descartado; status e metadados da decisão são persistidos. Reconhecimento facial não está implementado.

O perfil precisa de curso, período, biografia e 3 a 10 tags distintas. O guard aplica PROFILE_INCOMPLETE e, nas operações marcadas, ACADEMIC_VERIFICATION_REQUIRED. ADMIN é isento da exigência acadêmica nas rotas correspondentes. A CNH é uma foto reencodificada e armazenada privadamente no PostgreSQL; não existe OCR ou validação de autenticidade/validade da habilitação.

O aplicativo usa access token Bearer e refresh token. No nativo, o refresh fica no SecureStore; na web, no localStorage. O guard intersecta os papéis do token do aplicativo com os atuais do banco: um papel novo pode exigir refresh/login; revogação deixa de autorizar imediatamente. O painel usa cookies, MFA, origem e CSRF, com sessão persistida de escopo ADMIN válida e não revogada. Bearer do aplicativo só é aceito nas operações administrativas explicitamente permitidas de revisão acadêmica.

O catálogo IAM contém seis permissões. MANAGE_ADMINS protege as rotas de gestão de administradores; o controller administrativo legado continua baseado em AdminRoute, sem aplicar individualmente as outras cinco permissões. Não se deve apresentar o catálogo como autorização granular de todas as ferramentas.

8. Comunicação e integrações

A API configura Helmet, CORS com credenciais, validação de DTOs e limites de requisição. Swagger /docs só é habilitado fora de produção. /health e /health/ready ficam fora do prefixo /api/v1.

O chat autentica o JWT no handshake e oferece conversation:join, conversation:send e conversation:message. A mensagem é persistida antes da transmissão. O histórico REST recupera mensagens após desconexão; leitura atualiza recibos e notificações. O limite é 100 eventos de conversa por usuário por minuto. A transmissão em várias instâncias exige adapter compartilhado, ausente na configuração atual.

Notificações gerais são consultadas por REST. O histórico no app independe dos canais externos. Preferências, categorias e Não perturbe em America/Sao_Paulo controlam entregas externas, que dependem de dispositivo e provedores configurados. O envio é agendado em memória após visibilidade da transação, sem fila durável; não há garantia de entrega exatamente uma vez. Não existe agendador implementado de aviso 30 minutos antes da partida.

O mapa exibe os pontos cadastrados, não a posição GPS contínua do motorista. MapLibre e o estilo OpenFreeMap são carregados externamente. SMTP, Twilio e Expo são integrações condicionais; a presença do código não confirma sua habilitação operacional.

9. Execução, publicação e qualidade

Node 24 é requisito do backend. O PostgreSQL local pode ser iniciado pelo docker-compose do backend. Após configurar um ambiente próprio: npm ci --include=dev, npm run prisma:generate, npm run prisma:deploy e npm run start:dev. prisma:deploy altera o banco apontado; utilizar somente o ambiente pretendido. O seed é opcional para desenvolvimento e não é passo de produção.

O Render declara build com instalação de dependências de desenvolvimento, geração do Prisma e compilação. O start usa npm run start:prod, sem aplicar migrações. A configuração de auto-deploy está desligada no YAML; não houve consulta ao Dashboard. A API requer DATABASE_URL e segredos JWT; produção também exige TWO_FACTOR_ENCRYPTION_KEY. Origens do app/painel, URL do painel e provedores devem corresponder ao ambiente real.

Os workflows declaram lint, typecheck, unitários e integrações de busca, reservas, segurança, admin e IAM, além de migração/backfill e verificações do frontend. São verificações disponíveis, não resultados executados nesta atualização documental. Metas de latência, carga, disponibilidade e acessibilidade ainda precisam de medição própria.

10. Limites e evolução

Permanecem fora da implementação: rastreamento GPS contínuo, reconhecimento facial, matching geoespacial por raio, painel de sustentabilidade/CO2, processamento de pagamentos e alertas agendados de partida. Avaliação, ocorrências e controles de início/conclusão/cancelamento existem na API, mas não possuem uma jornada completa correspondente nas telas atuais do app. Publicidade é demonstrativa, sem gestão comercial de campanhas.

11. Rastreabilidade

Fontes: UniMove-front/package.json, App.tsx, src/config.ts, src/auth/tokenStorage*, src/components/OpenFreeMap.tsx; UniMove-back/package.json, src/main.ts, src/app.module.ts, src/common/guards/*, src/modules/{auth,verification,vehicles,rides,ride-requests,matching,chat,notifications,admin,users}/*; prisma/schema.prisma, prisma/migrations/*; render.yaml; .github/workflows/*.yml. Caminhos relativos à raiz do repositório.